6. Convolutional Codes

Convolutional codes have good error correcting capabilities, however, the generated codes are not systematic, and their decoding is relatively complex. They assign a single code word for an input channel symbol sequence of arbitrary length (like the arithmetic code in source coding), so there is no block creating step. Therefore, the already mentioned concepts used in describing the block codes and the corresponding methods are just partially applicable for convolutional codes.

6.1. Generating Convolutional Codes

Generating convolutional codes is done in a similar way how basic cyclic codes are produced: the input is considered as a polynomial; the code is generated by multiplication with generator polynomials, but in this case, more than one generator polynomial is used simultaneously. Thus the encoder device is a so called FIR encoder (finite impulse response encoder; also called as convolver), like the one shown in Figure 6.1.

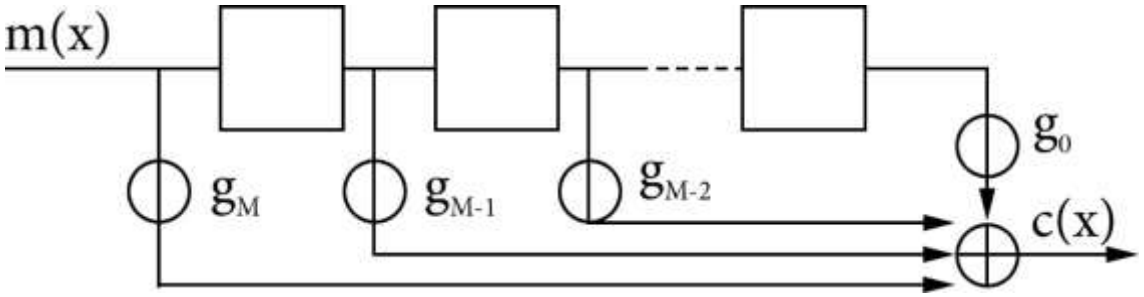


Figure 6.1: *Outline of a polynomial multiplier circuit*
g_i – polynomial coeff.; M – shift register's depth, i.e. number of shift registers; other elements same as in Figure 5.1

The circuit in Figure 6.1 multiplies its input, an m(x) polynomial of arbitrary length, by a g(x) polynomial, and the result is its output c(x). One coefficient is processed in a time segment. The convolutional encoder multiplies the input with different polynomials at the same time by using the same shift register (a multiple input XOR gate and an output for each multiplier polynomial). So the code is defined by the multiplier polynomials.

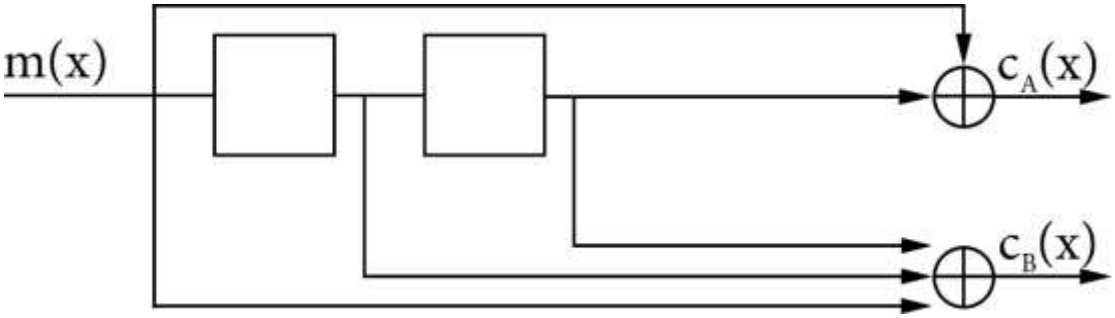


Figure 6.2: *Convolutional encoder with 2 outputs*

The multiplier polynomials of the convolutional code generated in Figure 6.2.:

$$g_A(x) = 1 + x^2$$
$$g_B(x) = 1 + x + x^2$$

In each time segment, a channel symbol enters the encoder, and for every output, one symbol exits. Hence, if we denote the number of outputs with N_0, then its net code rate is

$$R_{net} = \frac{1}{N_0}$$

If the input m(x) channel symbol sequence is not infinite, but is of length K, then the real code rate is even lower, as we need to wait M instances for the register to flush:

$$R_{eff} = \frac{K}{KN_0 + MN_0} = \frac{R_{net}}{1 + \frac{M}{K}} < R_{net}$$

The generated code can also be given in a vector form:

$$c_A(x) = m(x)g_A(x),\ c_B(x) = m(x)g_B(x), \text{ from this } \boldsymbol{c}(x) = m(x)\boldsymbol{g}(x)$$

An important property of convolutional codes is linearity that requires that the sum of two valid code words is also a valid code word. Based on the previous statement, this is also met:

$$\boldsymbol{c}_1(x) = m_1(x)\boldsymbol{g}(x)$$
$$\boldsymbol{c}_2(x) = m_2(x)\boldsymbol{g}(x)$$
$$\boldsymbol{c}_1(x) + \boldsymbol{c}_2(x) = \big(m_1(x) + m_2(x)\big)\boldsymbol{g}(x)$$

6.2. The State Machine Modell

For decoding the generated code, we have to know in which inner state the encoder is for any specific time instant. This can be achieved by considering the encoder as a state machine and using its **state diagram**. For this, the state labels are associated with the contents of the shift register, e.g. for the code in Figure 6.2:

$$S_0 \rightarrow 00$$
$$S_1 \rightarrow 10$$
$$S_2 \rightarrow 01$$
$$S_3 \rightarrow 11$$

A state diagram shows the output and the next state the system will be in, for each input received:

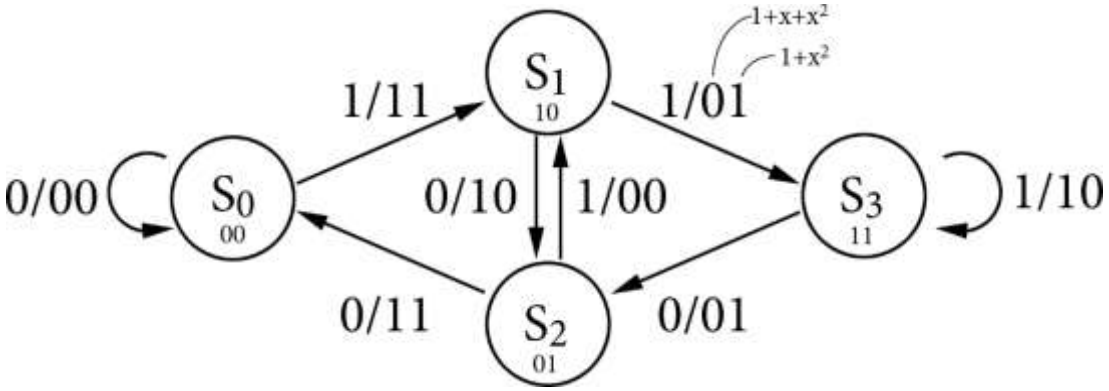


Figure 6.3: *The state diagram of encoder in Figure 6.2*

From these, it is easy to understand the following properties of convolutional codes:

- An all 0 input will create an all 0 output.
- Input 0's after at most M steps lead back to state 0…0.
- To every state, we can get from exactly two states, which have a different LSB.
- From every state, we can get to exactly two states, which have a different MSB.

An *alternative path* or adversary path in a convolutional code is a sequence of state transitions (and the corresponding output) of arbitrary length, that starts in state 0…0, leaves it at least once, and returns to this state. The d_f **free code distance** is the minimum of Hamming distances between the outputs of all possible alternative paths. Similarly to the code distance defined for linear block codes the free distance of a convolutional code is the basis of error correction. As the code is linear, based on Section 4.2, it is enough to find the alterative path with the output of smallest weight to get this free code distance; the weight is the free distance. In many cases, this can be easily found by just taking a look at the state diagram, e.g. in Figure 6.3, it is the path $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_0$, so $d_f = 5$.

The Bit Error Rate

In case of convolutional codes, we cannot talk about block decoding errors in the same way as for linear block codes, as we do not code blocks. Instead, we introduce **bit error rate P_b** as the probability of one binary symbol (bit) being incorrect in a symbol sequence that was decoded and corrected (in a way discussed later on). For the bit error rate, there can be given both a lower and an upper bound, based on transfer function T(N, J, D) and free code distance, as a function of channel error probability. So the transfer function, in fact, fully defines the error correcting capabilities of the code. (More details in the corresponding literature.)

For constructing convolutional codes with good error correcting properties, however, only a few methods are known. In order to find good codes, computers are used. Some examples for such codes can be found in the table below, for others please see [1].

R_{net}	M	Generator polynomials[17]	d_f
1/3	3	(13, 15, 17)	10
1/3	5	(47, 53, 75)	13
1/2	5	(53, 75)	8
1/4	4	(25, 27, 33, 37)	16

6.3. Decoding and Correcting in One Step: the Viterbi Algorithm

This algorithm is used at the receiver to obtain the estimate of message m(x) from a received code word of arbitrary length. As the code is not systematic, this task is far from trivial.

The algorithm uses as many processors as many states are in the state diagram (2^M). These are assigned to the states and are connected accordingly to the graph, i.e. each processor has two input and two output connections (which might, of course, lead into the same unit). In each time segment, each processor receives its input, the N_0 number of channel symbols.

Every processor has a register for storing one integer number, that contains the current value of the **likelihood metric** μ. (There are several types of likelihood metrics, for now, we will use Hamming distance.) Each processor also has a path register *pr* that can store a record of the decoded message bits (0 / 1 / X - empty) for the current path.

The decoder is operating continuously, so we suppose that the code is of infinite length, but at the start, the encoder is considered to be in state S_0. At the beginning, when receiving the first input of N_0 channel symbols, for processor S_0 $\mu = 0$ and for the others $\mu = \infty$, while all path registers contain only empty (X) symbols.

Then in each cycle, the processors execute the following tasks:

1. Send the value of the current path register and the likelihood metric to the output processors, and receive those that arrive from the source processors (pr_1, pr_2, μ_1, μ_2), and read the current input (N_0 channel symbols).

[17] The same octal notation used for cyclic codes

2. For each incoming path, calculate the difference (Hamming distance) between the actual input read and what it should be (based on the state diagram) for the given path leading to the processor. In the previous example, for $S_0 \rightarrow S_1$, transition the appropriate code is 10, but if the received is 11, then this value will be 1.
3. Add the resulting two distances to the corresponding received likelihoods (μ_1, μ_2) as a penalty, and select the smaller one[18]. This also selects which path (/previous state) is to be accepted. If the two are equal, select randomly.
4. Update the (state's) likelihood metric with the selected value, and set the path register to the received path. From the state diagram, write the input (message bit) that corresponds to the selected state to the end of the path register (into the first X from left). In our example, to transition $S_1 \rightarrow S_2$ this message bit is 0.

The controller that uses the decoder compares the contents of the path registers at the end of each time segment/cycle. If for *all processors*, on one or more positions (read from left to right) the same message bits are stored, then these matching bits are considered to be decoded and corrected, and are shifted out (to the left) from each path register.

It can be proved, that the algorithm follows the maximum-likelihood principle, i.e. from the possible inputs it finds the one with highest probability of corresponding to the given code of arbitrary length. The value of the likelihood metric in a given time segment for a specific processor is nothing else, but the total distance between the received code (the input), that may contain errors, and the code that belongs to the path starting in state S_0 and leading to the processor, based on the state diagram. So it shows *how likely is* that the estimate for the given path is the same as the original message sent – this is why the expression "likelihood metric" is used.

6.4. Improvements on Convolutional Codes

In this section, some of the problems of the previously introduced basic algorithm are discussed, with their possible solutions.

The Length of Path Registers
In real life hardware implementations (VLSI), obviously, the path registers are of finite length (e.g. 32 bit registers), so selecting their size properly is an important aspect in design. If all path registers are full, but they differ even in the first bit position, we have no place to write the estimate of the next cycle. In such cases, the processor with the best likelihood value is selected, its register's first bit is removed as the first decoded bit, and all registers are shifted left with one position.

Overflow in the Likelihood Metric
For a code long enough, that has many errors, an arithmetic overflow might occur in the register of the likelihood metric. To avoid this, the worst likelihood is inspected in every cycle. If it approaches the highest representable number, then we decrease every processor's likelihood value with the best (smallest) likelihood, as only their *difference* is important in the algorithm. (Of course, the previously mentioned additional meaning of the likelihood metric is lost this way.)

Improving Communication (the Traceback Method)
Another implementation related problem, for codes of greater depth, is the high number of processors (2^M) and their communication need (sending the path registers, mainly). This can be avoided by using the so called *traceback version* of the algorithm. The concept of this is that the LSBs of the state codes for two processors leading into the same processor, are different.

[18] The larger the Hamming distance is, the further we are from the message possibly sent (so actually, in this case, Hamming distance is more like a sort of an "unlikelihood metric").

So in this version, only the likelihood metric is sent in every cycle, while in the path register the LSB of the selected processor is written. This way, in a given time segment, it is still possible to trace the path that leads to the specific processor by checking the path register. But comparing the processors' results cannot be done by checking the fronts of their registers. Instead, a transition sequence is considered decoded where paths from every processor meet.

Elimination of Ties

As the Viterbi algorithm selects randomly in case of two equal estimates, it might happen that the one it will select is not the better one. This can be avoided by forwarding the sampled signal of the channel's binary symbol to the receiver's decoder without quantization (N_0 real numbers in each cycle). Obviously, for such values Hamming distance cannot be used (in the algorithm's 2nd step), so instead, an Euclidean distance is calculated. For example, if +5V is the ideal "1" on the channel and 0V is the ideal "0", and in a given cycle (of a code with rate 1/2) the code based on the state diagram should be 10, and the received values are 3.4V and 2.9 then the penalty is

$$p = (5 - 3.4)^2 + (0 - 2.9)^2$$

Naturally, in this case the likelihood metric is not an integer, but a real valued number. Because the quantization step is left out, this version is referred as the one that has a **soft-decision decoder**. This method does not only eliminate the possibility of ties (two real values are almost never identical), but it can also provide a better bit error probability by avoiding the information loss caused by quantization.

Increasing code rate

The codes discussed so far had a code rate of 1/2 at most. For less noisy channels, the need for better rates is present, which can be satisfied by a technique called "**puncturing"** a code. Generating these codes is done almost in the same fashion as in the basic version, but the time segments are grouped into periods, and in such a period only a part of the generated code is sent to the channel. This way, good code rates can be achieved. We use the (7, 5) code from the previous example and its punctured version with a puncturing period of 3 as an example. In the 2nd time segment of a period, only the output polynomial of 7 is forwarded, while in the 3rd, only the one that corresponds to 5 is sent.

Time segments	1st	2nd	3rd	4th	5th	6th
Periods	1st period			2nd period		
Message bits	1	1	0	1	0	0
(7, 5) basic code, R=1/2	11	01	01	00	10	11
(7,5), 7,5 punctured code, R=3/4	11	0	1	00	1	0

The state diagram of a punctured code consists of P times more states than the one for its "original" version, if the length of the period is P and the number of processors is still 2^M in the decoder. However, these processors have more complex programs as they have to "know" in which segment of the period they are, to calculate the penalties accordingly.

By puncturing a code, the code rate can be increased, but this results in a loss of error correcting capacity. Some examples of such punctured codes can be found in the table below (it is worth to compare them to the parameters of their non-punctured versions). For more examples please refer to [1].

R_{net}	M	Generator polynomials[19]	d_f
2/3	3	(13, 15) 17	4
3/4	5	(61, 53), 53, 53	5

[19] The same octal notation used for cyclic codes

4/5	5	(61, 53), 47, 47, 53	4
5/6	4	(37, 23), 23, 23, 25, 25	4

Acknowledgement

I thank *Miklós Vassányi* for his guidance in creating the epistemology overview, *Balázs Gaál* and *András Nemetz* for their contribution in editing and for their help with the equations and figures. I also thank *Benedek Szakonyi* for his tremendous help in translating and editing this syllabus.

Appendices

A1 Uniform source distribution maximises source entropy

Proof:

Let A = {p_0, p_1, …} and |A| = M. The proof can be considered complete once we have shown that H(A) – log(M) ≤ 0 and equality is satisfied only if $p_i = \frac{1}{M}$, as log(M) is the entropy of a source with equal distribution.

It can be rewritten as:

$$\sum_i p_i \log \frac{1}{p_i} - \log(M) =$$

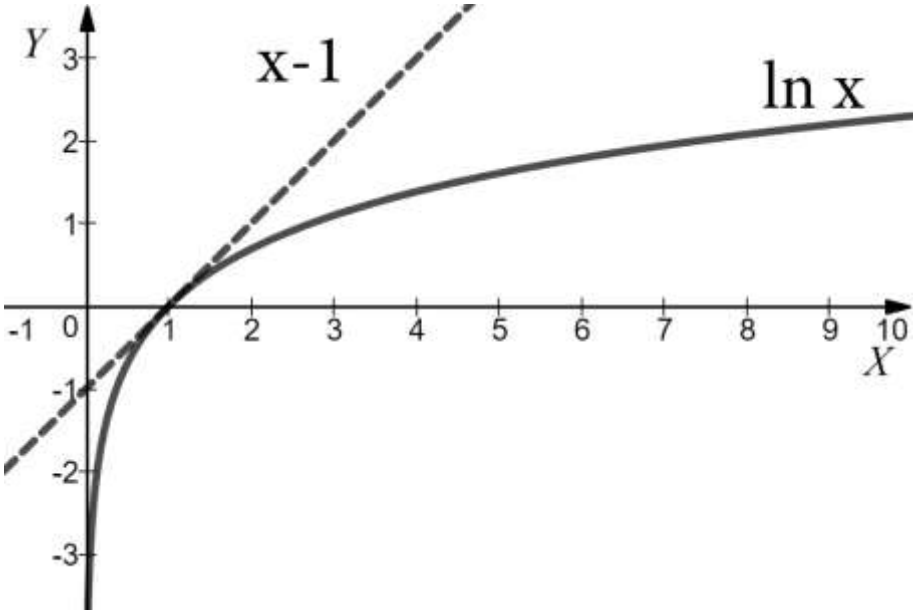
$$= \sum_i p_i \log \frac{1}{p_i M} =$$

$$= \frac{1}{\ln 2} \sum_i p_i \ln \frac{1}{p_i M}$$

Here we can use that by substituting $x = \frac{1}{p_i M}$ into all members of the summation, the following inequality is applicable:

$$\ln x \leq x - 1 \text{ and it is equal only if } x = 1$$

This inequality can be proved by analysing the function graphs



Substituting the inequality into the expression:

$$H(A) - \log M \leq \frac{1}{\ln 2} \sum_i p_i \left(\frac{1}{p_i M} - 1 \right) = 0$$

So indeed, H(A) – log(M) ≤ 0 and equality is satisfied only if $x = 1$, i.e. $p_i = \frac{1}{M}$ for all i, which is the case of equal distribution.

A2 "**There is no** lossless compression algorithm that could *always* encode a binary source's symbol sequence of arbitrary length N by using fewer symbols than N (not even by 1 less, i.e. with at most N-1 binary symbols)."

Proof:

The statement uses the pigeonhole principle. Suppose indirectly that there exists such an encoding machine. It would have 2^N different binary inputs of length N in total, where the number of possible outputs is 2^{N-1} if the output is of length N – 1, 2^{N-2} if its length is N – 2 and so on. Besides that any of these could be used, the total number of possible outputs would be still less than the number of possible inputs:

$$2^{N-1} + 2^{N-2} + \cdots + 2^1 = 2^N - 1 < 2^N$$

This means that there would exist at least two inputs that for the encoder produces the same output, so the code would not be decodable.